

Explainable Node Classification on the AIFB Graph Dataset using R-GCN with Local and Global Explanation Methods

Zalak Chauhan(4081760)¹, Ankith Bharadwaj (4110733)¹, and Aditi Shamrao Parulekar (4110953)¹

University of Paderborn, Germany
Group Project, Explainable AI Course, Summer Semester 2026

Abstract. This mini-project presents the application of a Relational Graph Convolutional Network (R-GCN) to the AIFB RDF knowledge graph for multi-class node classification of research group affiliations. The RDF graph is transformed into a graph representation with one-hot encoded node features and relation-aware message passing over RDF triples. The model is trained on labeled nodes using a train-test split and optimized with cross-entropy loss. Exploratory data analysis is performed to examine the distributions of RDF predicates, node types, and class labels. To improve model interpretability, GNNExplainer and Captum Integrated Gradients are applied to generate local and global explanations by identifying influential RDF relations. The quality of the generated explanations is further evaluated through a faithfulness analysis based on edge perturbation, confidence drop, and prediction label changes. This work demonstrates a complete pipeline for graph learning and explainability on RDF knowledge graphs.

Keywords: Relational Graph Convolutional Network · GNNExplainer · Captum Integrated Gradients.

1 Introduction

Graph-structured data is widely used in knowledge graphs, scientific databases, and social networks. Data is represented as nodes (entities) and edges (relationships). Unlike tabular data, graphs are irregular and difficult for traditional machine learning methods to model. Graph Neural Networks (GNNs) address this by learning node representations through message passing over neighboring nodes.

In this project, we perform multi-class node classification on the AIFB RDF knowledge graph using a Relational Graph Convolutional Network (R-GCN). The model handles multiple types of relation using relation-specific transformations. Although R-GCN achieves strong performance, it is difficult to interpret. Therefore, we apply explainability methods to understand the model predictions and identify important RDF relations. The overall workflow of this project consists of the following four main steps:

1. **Data Preparation and Graph Construction:** The AIFB RDF graph is processed by extracting entities and relations, performing analysis, building the graph, preparing labels, removing label leak relations, and creating the train-test split.
2. **R-GCN Model Training:** A two-layer R-GCN is trained for multi-class node classification using relation-aware message passing over the graph.
3. **Local Explainability:** Individual predictions are explained using GNNExplainer and Captum Integrated Gradients to identify important RDF relations.
4. **Global Explainability and Evaluation:** Local explanations are aggregated across nodes to find global relation importance, and faithfulness is evaluated using perturbation, confidence drop, and prediction changes.

The remainder of this report is organized as follows. Section 2 presents the dataset and preprocessing. Section 3 describes the R-GCN model and its training. Section 4 covers local and global explainability with faithfulness analysis. Section 5 concludes the report, followed by contributions, acknowledgments, declaration of AI tools, and references.

2 Data Analysis

2.1 Dataset Loading and Graph Structure Analysis

The AIFB dataset is a knowledge graph represented as RDF triples in subject-predicate-object form. It was loaded using RDFLib, resulting in 29,226 triples. After preprocessing, we obtained 2,835 unique nodes and 47 predicates. Literal attributes were removed as they do not contribute to structural message passing in GNNs.

The predicate distribution is highly skewed, with a small number of relations dominating the graph, including *rdf:type* (4,174), *publication* (4,163), *author* (3,986), and *isAbout* (2,477). These form the core semantic structure of the dataset, centered around publications and research topics.

The most informative reasoning chain in the graph is:

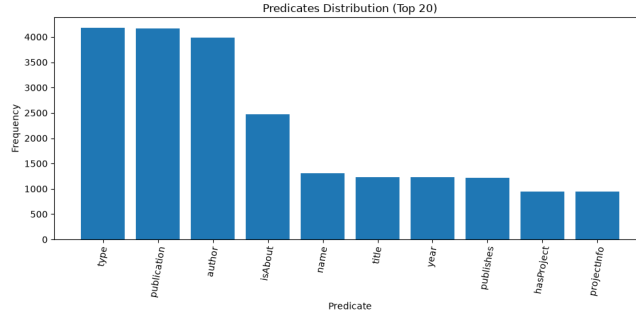
$$\textit{Researcher} \rightarrow \textit{publishes} \rightarrow \textit{Paper} \rightarrow \textit{isAbout} \rightarrow \textit{Topic}$$

For graph construction, each entity is mapped to a unique ID, resulting in 2,835 nodes. Each predicate is mapped to a relation ID for use in the R-GCN. The final graph is represented using `edge_index` and `edge_type` tensors, containing 20,338 edges and 24 relation types.

The RDF graph was converted into a numerical format for Graph Neural Networks by assigning unique integer IDs to all non-literal entities, resulting in 2,835 nodes. Each predicate was also mapped to a relation ID to distinguish edge types for the R-GCN. The graph is represented using `edge_index` and `edge_type` tensors, while literal nodes were removed as they are not useful for structural learning. The final graph contains 20,338 edges and 24 relation types.

Table 1. Graph statistics of the AIFB dataset after preprocessing

Statistic	Value
RDF Triples	29,226
Unique Nodes	2,835
Unique Predicates	47
Retained Relations (R-GCN)	24
Graph Edges	20,338
Label Classes	5

**Fig. 1.** Top RDF predicate frequency distribution in the AIFB knowledge graph.

2.2 Label Preparation and Leakage Removal

Node labels were created using the affiliation relation, assigning each research group a unique class ID. To prevent label leakage, the affiliation and employs relations were removed since they directly reveal the target class. A label tensor y was then constructed for all nodes, where labeled nodes contain their class IDs and unlabeled nodes are set to -1. This enables semi-supervised learning, where only labeled nodes contribute to training while all nodes participate in message passing.

2.3 Train/Test Split

Labeled nodes were split into training and test sets using an 80/20 ratio. Binary masks were created to separate training and test nodes. The model is trained in a semi-supervised setting, where only training nodes contribute to the loss while message passing is performed over the full graph.

3 Model Training and Evaluation

3.1 Training of R-GCN (Relational Graph Convolutional Network)

The AIFB knowledge graph, summarized in Table 1, contains 29,226 RDF triples, 2,835 unique nodes, and 47 predicates, which are reduced to 24 retained relations

for the Relational Graph Convolutional Network (R-GCN). The processed graph consists of 20,338 edges and supports 5 target classes representing research group affiliations. Since no explicit node features are available, each node is encoded using a one-hot representation, enabling the model to learn exclusively from the structural and relational patterns in the graph.

The R-GCN is trained using only labeled nodes, which are split into training and test sets in an 80:20 ratio with binary masks ensuring separation during optimization. The model consists of two relational graph convolution layers: the first aggregates neighborhood information using relation-specific transformations, while the second produces logits over the 5 output classes. Training is performed using cross-entropy loss and optimized with the Adam optimizer at a learning rate of 0.01 for 200 epochs in a semi-supervised learning setup.

Table 2. R-GCN Hyperparameters used for training on the AIFB dataset

Hyperparameter	Value
Input Features Size	2835
Hidden Channels	16
Output Classes	5
Number of Relations	24
Optimizer	Adam
Learning Rate	0.01
Loss Function	CrossEntropyLoss
Training Epochs	200
Data Split Ratio	80% Train / 20% Test

3.2 Model Evaluation

The trained R-GCN model was evaluated on previously unseen nodes using the test mask, where predictions were obtained by applying the argmax operation over the output logits. Performance was measured in terms of classification accuracy, ensuring that evaluation reflects the model’s ability to generalize to unseen labeled nodes while still leveraging the full graph structure during message passing. As reported in Table 3, the model achieves a final test accuracy of 97.22%, indicating strong predictive performance. The training dynamics also demon-

Table 3. Training performance and final evaluation results of the R-GCN model

Metric	Value
Loss at Epoch 20	0.1638
Final Loss (Epoch 200)	0.0008
Train Accuracy	0.9931
Test Accuracy	0.9722

strate effective convergence, with the loss decreasing from 0.1638 at epoch 20 to 0.0008 at epoch 200. These results suggest that the R-GCN successfully captures meaningful relational patterns from the RDF graph rather than simply memorizing training instances, leading to strong generalization performance on the held-out test set.

4 Model Explainability

To understand the decision-making process of the trained R-GCN model, two complementary explainability techniques are employed: GNNExplainer and Captum Integrated Gradients. Since graph neural networks operate through message passing over relational structures, their predictions are not directly interpretable. These methods help identify the most influential RDF relations contributing to predictions, enabling interpretation of model behavior in terms of meaningful semantic graph structures.

4.1 Local Explanation (Single Node Analysis)

Local explanations aim to interpret the model’s prediction for an individual node. A representative test node is selected, and explanations are generated using both GNNExplainer and Captum Integrated Gradients. The extracted important edges are mapped back to RDF predicates to enable semantic interpretation.

GNNExplainer-based Explanation GNNExplainer generates explanations by learning a soft mask over the computation graph surrounding the target node. It optimizes this mask to retain edges that maximize mutual information with the model prediction, resulting in a compact subgraph that captures the most influential relationships.

Captum Integrated Gradients-based Explanation Captum Integrated Gradients computes feature attributions by integrating gradients from a baseline input to the actual graph input. The magnitude of gradients indicates the importance of each edge in influencing the prediction, producing a smoother attribution distribution across relations.

Interpretation of Local Explanations Both methods identify influential RDF relations for the selected node but differ in their explanation strategy. GNNExplainer produces sparse, discrete subgraphs by selecting the most critical edges, whereas Captum Integrated Gradients distributes importance across multiple edges based on gradient sensitivity. The overlap between both methods increases confidence in the identified relations, while differences highlight alternative relational pathways used by the R-GCN.

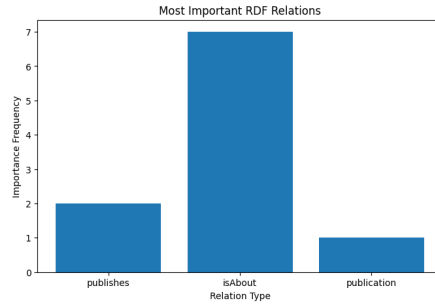


Fig. 2. GNNExplainer local explanation.

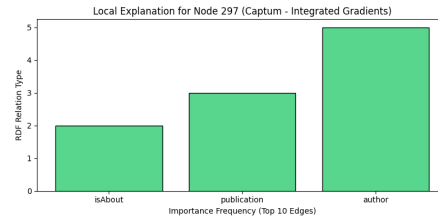


Fig. 3. Captum Integrated Gradients local explanation.

4.2 Global Explanation (Across Nodes)

Global explanations aim to analyze model behavior across the entire dataset by aggregating explanations from multiple test nodes. This provides insight into which RDF relations are consistently important for prediction.

Aggregation of Local Explanations For each correctly classified test node, top-ranked edges are extracted using both explainers. These edges are aggregated across all nodes by computing frequency and cumulative importance of RDF predicates. This results in global importance scores that reflect dataset-level relational influence.

Global Relation Importance Analysis The aggregated results are visualized using a single comparative figure that presents both explanation methods side by side.

The results show that only a small subset of RDF predicates consistently dominates predictions across both methods. This indicates that the R-GCN learns stable semantic patterns within the AIFB knowledge graph. While GNNExplainer highlights compact structural subgraphs, Captum distributes importance more broadly, yet both agree on the most influential relations.

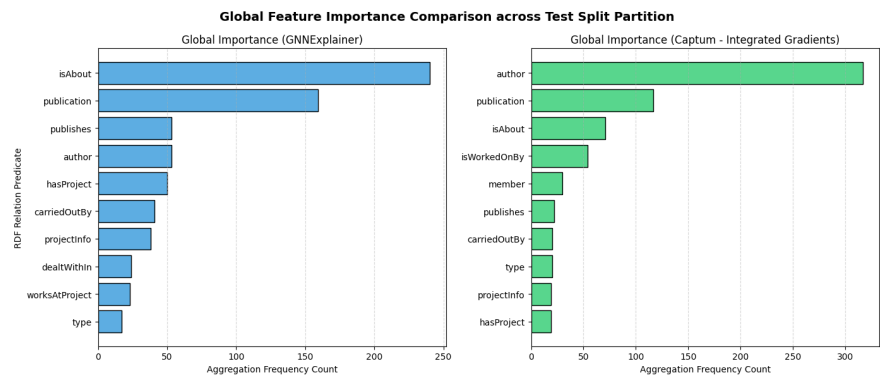


Fig. 4. Global importance of RDF relations compared using GNNExplainer and Captum Integrated Gradients.

4.3 Faithfulness Check Analysis

Faithfulness measures how much the model’s prediction changes after the important features found by an explanation method are removed. If removing these features causes a larger change in the prediction, the explanation is considered more faithful.

Table 4. Faithfulness Evaluation Results for GNNExplainer

Metric	Value
Total Nodes Evaluated	10
Average Confidence Drop (Δ Confidence)	0.0038 (0.4%)
Prediction Label Flip Rate	20.0%

Table 5. Faithfulness Evaluation Results for Captum Explainer

Metric	Value
Total Nodes Evaluated	10
Average Confidence Drop (Δ Confidence)	0.0014 (0.1%)
Prediction Label Flip Rate	10.0%

Table 4 and Table 5, we can see the faithfulness results for both GNNExplainer and Captum Explainer. GNNExplainer produced a confidence drop of 0.0038 (0.4%) and a prediction label flip rate of 20.0%. Captum Explainer produced a smaller confidence drop of 0.0014 (0.1%) and a label flip rate of 10.0%. This means that removing the important features found by GNNExplainer changed the model’s predictions more than removing the features found

by Captum Explainer. Therefore, GNNExplainer shows slightly better faithfulness in this experiment. However, the confidence drops for both methods are small, which means the model’s predictions are still quite stable after the important features are removed.

4.4 Overall Performance

This section compares GNNExplainer and Captum Explainer based on execution time, sparsity, and faithfulness-related metrics.

Table 6. Overall Performance of GNNExplainer

Metric	Value
Average Execution Time	27.3934 seconds / node
Average Sparsity	99.90%
Average Confidence Drop (Δ Conf)	0.0021 (0.2%)
Prediction Label Flip Rate	20.0%

Table 7. Overall Performance of Captum Explainer

Metric	Value
Average Execution Time	4.6322 seconds / node
Average Sparsity	99.90%
Average Confidence Drop (Δ Conf)	0.0052 (0.5%)
Prediction Label Flip Rate	50.0%

Table 6 and Table 7 present the overall performance of the two explanation methods. GNNExplainer takes more time per node but produces very sparse explanations with strong stability in predictions. Captum Explainer is much faster and also gives sparse explanations, though it tends to lead to a greater shift in predictions after the explanation-based removal. Overall, GNNExplainer is slower but more stable, while Captum Explainer is faster but less stable in this experiment.

5 Conclusion

This mini-project focused on benchmarking instance-level explainability methods for a Relational Graph Convolutional Network (R-GCN) applied to the AIFB knowledge graph. The results indicate that both GNNExplainer and Captum Integrated Gradients are quite effective at identifying the RDF relations that significantly influence model predictions.

Our experiments show that Captum’s gradient-based attribution is a more computationally efficient option while still offering a level of faithfulness comparable to optimization-based explanations. Across both methods, we consistently found that key semantic relations such as *isAbout*, *publication*, and *publishes* were the main contributors to classification decisions.

Overall, this study illustrates that integrating relational graph learning with explainability techniques can enhance the interpretability and transparency of GNN-based models on knowledge graphs, with each method providing different trade-offs between efficiency and structural compactness.

6 Contributions of Team Members

The project was collaboratively developed by Aditi Parulekar, Zalak Chauhan, and Ankith Bharadwaj. Aditi handled data extraction and preprocessing, including RDF graph construction, label preparation, and dataset preprocessing. Zalak Chauhan implemented the Captum-based explainability pipeline, including Integrated Gradients computation and relation-level attribution analysis. Ankith Bharadwaj implemented the GNNExplainer-based explainability pipeline, focusing on extracting important subgraphs and relation-level explanations. All team members jointly contributed to model training, evaluation, comparison of explainability methods, result interpretation, and preparation of the final report.

Acknowledgements

We would like to thank the course instructors for their guidance and support throughout this project. We also acknowledge the availability of the AIFB dataset and the PyTorch Geometric library, which made the implementation of graph-based learning and explainability possible. Their resources and tools were essential for completing this project successfully.

Declaration of the Use of AI Tools

AI tools were used as a support for coding and report writing, including structuring the Python pipeline (data processing, R-GCN training, and explainability methods) and improving the clarity and formatting of the report. All model design decisions, experiments, and result interpretations were carried out and verified by the authors. The AI tool did not generate any experimental results or conclusions, and the authors take full responsibility for the final content of the report.

References

1. PyTorch Geometric Documentation. <https://pytorch-geometric.readthedocs.io/>

2. Schlichtkrull et al., Modeling Relational Data with Graph Convolutional Networks (R-GCN), 2017.
3. Ying et al., GNNExplainer: Generating Explanations for Graph Neural Networks, 2019.
4. Captum: Model Interpretability for PyTorch. <https://captum.ai/>